

# Microservices Assignment

Project README / Submission Documentation

## Project Overview

This project demonstrates a simple microservices architecture using **Spring Boot**, **OpenFeign**, **Docker**, and **PostgreSQL**.

The project consists of two microservices: **Client Service**, which receives requests and communicates with the Storage Service using OpenFeign, and **Storage Service**, which handles file operations and stores data in PostgreSQL.

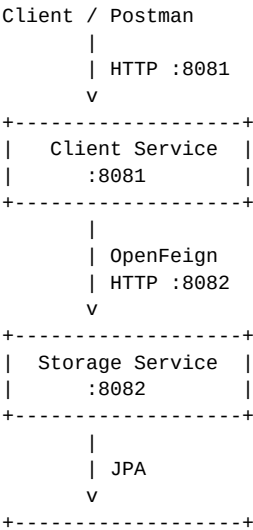
## Project Structure

```
IdeaProjects/
├── client-service/
│   ├── src/
│   ├── pom.xml
│   └── Dockerfile
├── storage-service/
│   ├── src/
│   ├── pom.xml
│   └── Dockerfile
├── docker-compose.yml
└── README.md
```

## Technologies Used

| Technology              | Version / Use              |
|-------------------------|----------------------------|
| Java                    | 17                         |
| Spring Boot             | 3.3.13                     |
| Spring Cloud OpenFeign  | Microservice communication |
| Spring Data JPA         | Database persistence       |
| PostgreSQL              | 16                         |
| Docker / Docker Compose | Containerization           |
| Maven                   | Build and dependencies     |
| Postman                 | API testing                |

## Architecture



```

|   PostgreSQL   |
|   :5432        |
+-----+

```

## Part 1 – Microservice Communication

The Client Service communicates with the Storage Service using **Spring Cloud OpenFeign**. The Client Service exposes the public API on port **8081**, while the Storage Service runs on port **8082**.

### File Download Endpoint

```
GET http://localhost:8081/client/download/{filename}
```

The request is forwarded by the Client Service to the Storage Service through OpenFeign.

```
GET /files/download/{filename}
```

## Part 2 – PostgreSQL and JPA

PostgreSQL is used to persist data received by the Storage Service. The Storage Service uses Spring Data JPA, Hibernate, and the PostgreSQL JDBC driver.

```

Database: storage_db
Username: postgres
Password: postgres
Port: 5432

```

The persistence entity is **StoredData**. The database table is **stored\_data** with the fields **id** and **content**.

## Part 2 – Data Flow

```

Postman
  |
  v
Client Service :8081
  |
  | OpenFeign
  v
Storage Service :8082
  |
  | Spring Data JPA
  v
PostgreSQL

```

### Request

```

POST http://localhost:8081/client/data

{
  "content": "Hello from Part 2"
}

```

### Example response

```

{
  "id": 1,
  "content": "Hello from Part 2"
}

```

# Running the Project

## Prerequisites

Java 17, Maven, and Docker Desktop should be installed.

Start the services from the project directory:

```
cd C:\Users\DELL\IdeaProjects
docker compose up -d
docker ps
```

Expected containers:

```
client-service
storage-service
postgres
```

## Service Ports

| Service         | Port |
|-----------------|------|
| Client Service  | 8081 |
| Storage Service | 8082 |
| PostgreSQL      | 5432 |

## Testing with Postman

### Test 1 – Download File

GET <http://localhost:8081/client/download/docker-test.txt>

### Test 2 – Save Data

POST <http://localhost:8081/client/data>

Use **Content-Type: application/json** and send:

```
{
  "content": "Hello from Part 2"
}
```

Expected response:

```
{
  "id": 1,
  "content": "Hello from Part 2"
}
```

## Database Verification

```
docker exec -it postgres psql -U postgres -d storage_db -c "SELECT * FROM stored_data;"
```

This command verifies that the data has been persisted in PostgreSQL.

## Docker Compose

Docker Compose starts client-service, storage-service, and postgres. The services communicate over Docker's internal network.

```
Client → http://storage-service:8082
Storage → jdbc:postgresql://postgres:5432/storage_db
```

## Stopping the Project

```
docker compose down
```

To remove the PostgreSQL volume as well:

```
docker compose down -v
```

Removing the volume deletes the stored PostgreSQL data.

## Requirements Checklist

| Requirement                   | Status    |
|-------------------------------|-----------|
| Two Spring Boot microservices | Completed |
| Client Service                | Completed |
| Storage Service               | Completed |
| OpenFeign communication       | Completed |
| REST APIs                     | Completed |
| Dockerfiles                   | Completed |
| Docker Compose                | Completed |
| PostgreSQL                    | Completed |
| Spring Data JPA               | Completed |
| Entity                        | Completed |
| Repository                    | Completed |
| Data persistence              | Completed |
| Postman testing               | Completed |
| Database verification         | Completed |

## Conclusion

This project demonstrates communication between two Spring Boot microservices using OpenFeign, containerization using Docker and Docker Compose, and persistent data storage using PostgreSQL and Spring Data JPA.